

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

CO5-AT1 Pseudocode Design

Course Code:	DSA0405	Course Title:	FUNDAMENTALS OF DATA SCIENCE
CO Assessed:	CO5-AT1 Pseudocode Design	Assessment:	CO5-AT1 Pseudocode Design
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	To understand the concept of supervised and unsupervised methods in machine learning		
Student Name:	B Tharun Kumar	Reg. No.:	192424356
Section / Batch:	AI DATA SCIENCE	Date:	25-08-2026

Code of Conduct

I B Tharun Kumar/192424356 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *B Tharun Kumar*

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	

Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

Pseudocode Assignment

1. CART Classification Algorithm using Gini Impurity

Problem Statement:

The Classification and Regression Tree (CART) algorithm builds a binary decision tree by recursively splitting the dataset into two child nodes at each step. The quality of a split is measured using Gini Impurity, which indicates how mixed the classes are within a node. For every feature and every possible threshold value, the algorithm calculates the weighted Gini Impurity of the resulting left and right subsets. The split that produces the lowest weighted Gini Impurity is chosen as the best split for that node. This splitting process is repeated recursively on each child node until a stopping condition is met, such as maximum depth or pure class nodes. The final result is a fully constructed binary classification tree that can be used to classify new data points.

Algorithm:

1. Load the training dataset containing features and class labels.
2. For each feature, evaluate every possible split threshold.
3. Calculate the Gini Impurity of the left and right subsets for each split.
4. Compute the weighted Gini Impurity and select the split with the minimum value as the best split.
5. Split the dataset into two child nodes using the best feature and threshold.
6. Repeat the process recursively on each child node until a stopping condition (pure node, max depth, or minimum samples) is reached, then assign leaf labels.

Pseudocode:

BEGIN CART_Classification

FUNCTION GiniImpurity(dataset):

 classes = distinct class labels in dataset

 total = number of records in dataset

 gini = 1

 FOR each class c in classes:

 p_c = count(records with label c) / total

 gini = gini - (p_c * p_c)

 RETURN gini

FUNCTION GiniSplit(left_subset, right_subset):

 n_left = size(left_subset)

 n_right = size(right_subset)

 n_total = n_left + n_right

 weighted_gini = (n_left/n_total)*GiniImpurity(left_subset) +
 (n_right/n_total)*GiniImpurity(right_subset)

 RETURN weighted_gini

FUNCTION FindBestSplit(dataset, features):

 best_gini = INFINITY

 best_feature = NULL

 best_threshold = NULL

 FOR each feature F in features:

 FOR each unique value V of feature F in dataset:

 left_subset = records where F <= V

```

    right_subset = records where  $F > V$ 
    IF left_subset is empty OR right_subset is empty:
        CONTINUE
    current_gini = GiniSplit(left_subset, right_subset)
    IF current_gini < best_gini:
        best_gini = current_gini
        best_feature = F
        best_threshold = V
    RETURN best_feature, best_threshold, best_gini

```

FUNCTION BuildTree(dataset, depth):

```

    IF all records in dataset belong to same class:
        RETURN LeafNode(class label)
    IF depth == max_depth OR size(dataset) < min_samples_split:
        RETURN LeafNode(majority class in dataset)
    feature, threshold, gini = FindBestSplit(dataset, features)
    IF feature is NULL:
        RETURN LeafNode(majority class in dataset)
    left_data = records where feature <= threshold
    right_data = records where feature > threshold
    left_child = BuildTree(left_data, depth + 1)
    right_child = BuildTree(right_data, depth + 1)
    RETURN DecisionNode(feature, threshold, left_child, right_child)

```

MAIN:

```

root = BuildTree(training_dataset, depth = 0)
RETURN root

```

END CART_Classification

Output:

Best Split Found: Feature = Income, Threshold = 50000

Weighted Gini Impurity = 0.28

Constructed Classification Tree:

Root -> [Income <= 50000]

Left -> [Age <= 30] -> Leaf: Class 'No'

Right -> [Age > 30] -> Leaf: Class 'Yes'

The tree correctly classifies training records with minimum overall Gini Impurity at each split.

2. Decision Tree for Customer Purchase Prediction

Problem Statement:

A retail company wants to predict whether a customer will purchase a product based on customer attributes. The available features for prediction are Age, Income, Previous Purchases, and Browsing History. A Decision Tree model is used because it can handle both numerical and categorical data while remaining easy to interpret. The tree is built by selecting, at each node, the feature that best separates customers who purchased from those who did not. The feature is chosen using a splitting criterion such as Information Gain or Gini Impurity, and the dataset is split accordingly. Once trained, the tree is used to classify new customers as either 'Will Purchase' or 'Will Not Purchase'.

Algorithm:

1. Collect and preprocess the customer dataset with features Age, Income, Previous Purchases, Browsing History, and label Purchased.
2. At the root node, calculate Information Gain (or Gini Impurity) for each feature.
3. Select the feature with the highest Information Gain as the splitting attribute.
4. Split the dataset into branches based on the values/ranges of the selected feature.
5. Repeat the splitting process recursively for each branch until all records in a branch belong to one class or a stopping condition is met.
6. Use the completed tree to traverse from root to leaf for a new customer and predict Purchased = Yes/No.

Pseudocode:

BEGIN Customer_Purchase_Prediction

INPUT: dataset with features [Age, Income, PreviousPurchases,
BrowsingHistory] and target label Purchased (Yes/No)

FUNCTION TrainDecisionTree(dataset, features, depth):

IF dataset is empty:

RETURN NULL

IF all records have same Purchased value:

RETURN LeafNode(Purchased value)

IF depth == max_depth OR features is empty:

RETURN LeafNode(majority class of Purchased)

best_feature = feature in features that gives maximum
Information Gain (or minimum Gini Impurity)
when splitting dataset

tree = DecisionNode(best_feature)

FOR each possible value/range V of best_feature:

subset = records in dataset where best_feature == V

IF subset is empty:

tree.addBranch(V, LeafNode(majority class of dataset))

ELSE:

remaining_features = features - {best_feature}

child = TrainDecisionTree(subset, remaining_features, depth+1)

tree.addBranch(V, child)

RETURN tree

FUNCTION Predict(tree, customer):

IF tree is LeafNode:

```
    RETURN tree.label
feature_value = customer[tree.feature]
next_node = tree.branch(feature_value)
RETURN Predict(next_node, customer)
```

MAIN:

```
features = [Age, Income, PreviousPurchases, BrowsingHistory]
model = TrainDecisionTree(training_dataset, features, depth = 0)

new_customer = {Age: 28, Income: 45000,
                 PreviousPurchases: 3, BrowsingHistory: High}
result = Predict(model, new_customer)
PRINT "Will customer purchase?", result
```

END Customer_Purchase_Prediction

Output:

Root Node Split: BrowsingHistory (highest Information Gain)

Decision Tree Structure:

BrowsingHistory = High -> PreviousPurchases > 2 -> Yes

BrowsingHistory = High -> PreviousPurchases <= 2 -> Income > 40000 -> Yes

BrowsingHistory = Low -> No

Prediction for New Customer

(Age=28, Income=45000, PreviousPurchases=3, BrowsingHistory=High):

Will customer purchase? -> Yes

3. Regression Analysis Workflow

Problem Statement:

Regression analysis is a statistical technique used to understand and model the relationship between a dependent variable and one or more independent variables. A complete regression workflow begins with loading a raw dataset from a file such as CSV or a database. The independent variables (predictors) and dependent variable (target/outcome) must be correctly identified from the dataset columns. The dataset is then cleaned, pre-processed, and split into training and testing sets before a regression model is trained. After training, the model is used to generate predictions on unseen test data. Finally, the model's performance is evaluated using metrics such as MAE, MSE, RMSE, and R-Squared to judge prediction accuracy.

Algorithm:

1. Load the dataset from the source file (CSV/Excel/Database) into memory.
2. Identify independent variables (X) and the dependent variable (Y) from the dataset columns.
3. Preprocess the data: handle missing values, encode categorical columns, and scale numerical features.
4. Split the dataset into training and testing sets (e.g., 80% train, 20% test).
5. Train a regression model on the training data using the fit() function.
6. Predict outcomes on the test data and evaluate the model using MAE, MSE, RMSE, and R-Squared.

Pseudocode:

BEGIN Regression_Analysis_Workflow

STEP 1: Load Dataset

```
dataset = READ_DATA(file_path)
```

STEP 2: Identify Variables

independent_variables (X) = all predictor columns in dataset

dependent_variable (Y) = target/outcome column in dataset

STEP 3: Preprocess Data

Handle missing values in dataset

Encode categorical variables if any

Normalize/Scale numerical features

STEP 4: Split Dataset

```
(X_train, X_test, Y_train, Y_test) = SPLIT(X, Y,  
train_ratio = 0.8)
```

STEP 5: Train Regression Model

```
model = RegressionModel()
```

```
model.fit(X_train, Y_train)
```

STEP 6: Make Predictions

```
Y_predicted = model.predict(X_test)
```

STEP 7: Evaluate Model

```
MAE = MeanAbsoluteError(Y_test, Y_predicted)
```

```
MSE = MeanSquaredError(Y_test, Y_predicted)
```

```
RMSE = SQRT(MSE)
```

```
R2 = RSquared(Y_test, Y_predicted)
```

```
PRINT "MAE:", MAE  
PRINT "MSE:", MSE  
PRINT "RMSE:", RMSE  
PRINT "R-Squared:", R2
```

STEP 8: Output Final Model

RETURN trained model and evaluation metrics

END Regression_Analysis_Workflow

Output:

Dataset Loaded: 500 records, 5 columns

Independent Variables: X1, X2, X3, X4

Dependent Variable: Y

Model Evaluation Results:

MAE = 2.34

MSE = 8.91

RMSE = 2.98

R-Squared = 0.87

The model explains 87% of the variance in the target variable, indicating a good regression fit.

4. Simple Linear Regression (House Price from Area)

Problem Statement:

Simple linear regression is used to model the relationship between one independent variable and one dependent variable using a straight line. In this problem, House Area is the independent variable (X) and House Price is the dependent variable (Y). The least-squares approach is used to find the line that minimizes the sum of squared differences between actual and predicted prices. This involves calculating the slope (m) and intercept (b) of the best-fit line using the means and deviations of X and Y. Once the slope and intercept are found, the regression equation $\text{Price} = m * \text{Area} + b$ can be used for prediction. The model is then evaluated using metrics like RMSE and R-Squared to check how well it fits the data.

Algorithm:

1. Collect the dataset containing House_Area (X) and House_Price (Y) values.
2. Calculate the mean of X (mean_X) and the mean of Y (mean_Y).
3. Compute the slope (m) using the least-squares formula based on deviations from the means.
4. Compute the intercept (b) using the calculated slope and the means of X and Y.
5. Form the regression equation: $\text{Price} = m * \text{Area} + b$.
6. Use the equation to predict prices for new area values and evaluate the model using RMSE and R-Squared.

Pseudocode:

BEGIN Simple_Linear_Regression

INPUT: dataset with House_Area (X) and House_Price (Y)

STEP 1: Load Data

X = list of house areas

Y = list of house prices

n = number of data points

STEP 2: Calculate Means

$\text{mean_X} = \text{SUM}(X) / n$

$\text{mean_Y} = \text{SUM}(Y) / n$

STEP 3: Calculate Slope (m) using Least Squares

numerator = 0

denominator = 0

FOR $i = 1$ to n :

 numerator = numerator + $(X[i] - \text{mean_X}) * (Y[i] - \text{mean_Y})$

 denominator = denominator + $(X[i] - \text{mean_X})^2$

$m = \text{numerator} / \text{denominator}$

STEP 4: Calculate Intercept (b)

$b = \text{mean_Y} - (m * \text{mean_X})$

STEP 5: Form Regression Equation

Price = $m * \text{Area} + b$

STEP 6: Predict Price for New Area

FUNCTION Predict(area):

 RETURN $m * \text{area} + b$

new_area = 1500

```
predicted_price = Predict(new_area)
PRINT "Predicted Price:", predicted_price
```

STEP 7: Evaluate Model

```
Y_predicted = [Predict(X[i]) for i = 1 to n]
RMSE = SQRT(SUM((Y[i]-Y_predicted[i])^2)/n)
R2 = RSquared(Y, Y_predicted)
PRINT "RMSE:", RMSE, "R-Squared:", R2
```

END Simple_Linear_Regression

Output:

Calculated Slope (m) = 250.5

Calculated Intercept (b) = 15000

Regression Equation: Price = 250.5 * Area + 15000

Predicted Price for Area = 1500 sq.ft: Rs. 3,90,750

Model Evaluation:

RMSE = 12500

R-Squared = 0.91

The high R-Squared value indicates a strong linear relationship between house area and house price.

5. Multiple Linear Regression (Student Final Mark)

Problem Statement:

Multiple linear regression extends simple linear regression by using more than one independent variable to predict a dependent variable. In this problem, the independent variables are Attendance, Internal Marks, Assignment Marks, and Study Hours, and the dependent variable is the Final Mark. The relationship is modelled as a linear equation with one coefficient for each independent variable plus an intercept term. The coefficients are estimated using the least-squares method, commonly implemented through the Normal Equation or gradient descent. Once trained, the model equation can be used to predict a student's final mark from their attendance and performance data. The model's accuracy is evaluated using metrics such as MAE, RMSE, and R-Squared on test data.

Algorithm:

1. Collect the dataset with independent variables (Attendance, Internal Marks, Assignment Marks, Study Hours) and dependent variable (Final Mark).
2. Add an intercept (bias) column of 1's to the independent variable matrix X.
3. Split the dataset into training and testing sets.
4. Estimate the regression coefficients using the least-squares Normal Equation: $B = (X\text{-transpose} * X)\text{-inverse} * X\text{-transpose} * Y$.
5. Form the multiple regression equation using the estimated coefficients b_0, b_1, b_2, b_3, b_4 .
6. Use the equation to predict final marks on test data and evaluate the model using MAE, RMSE, and R-Squared.

Pseudocode:

BEGIN Multiple_Linear_Regression

INPUT: independent variables

X1 = Attendance, X2 = Internal_Marks,

X3 = Assignment_Marks, X4 = Study_Hours

dependent variable Y = Final_Mark

STEP 1: Load and Prepare Data

dataset = READ_DATA(file_path)

X = matrix of [X1, X2, X3, X4] for all students

Y = vector of Final_Mark for all students

n = number of students (rows)

STEP 2: Add Bias/Intercept Term

Add a column of 1's to X (for intercept term b0)

X = [1, X1, X2, X3, X4]

STEP 3: Split Dataset

(X_train, X_test, Y_train, Y_test) = SPLIT(X, Y, 0.8)

STEP 4: Estimate Coefficients using Least Squares

// Normal Equation: $B = (X^T * X)^{-1} * X^T * Y$

B = MATRIX_INVERSE(TRANSPOSE(X_train) * X_train)

* TRANSPOSE(X_train) * Y_train

// B = [b0, b1, b2, b3, b4]

STEP 5: Form Regression Equation

Final_Mark = b0 + b1*Attendance + b2*Internal_Marks

$$+ b3*Assignment_Marks + b4*Study_Hours$$

STEP 6: Predict Final Marks

FUNCTION Predict(attendance, internal, assignment, study_hrs):

 RETURN b0 + b1*attendance + b2*internal

 + b3*assignment + b4*study_hrs

Y_predicted = Predict(X_test values)

STEP 7: Evaluate Model

MAE = MeanAbsoluteError(Y_test, Y_predicted)

MSE = MeanSquaredError(Y_test, Y_predicted)

RMSE = SQRT(MSE)

R2 = RSquared(Y_test, Y_predicted)

PRINT "MAE:", MAE, "RMSE:", RMSE, "R-Squared:", R2

STEP 8: Output Final Model

 RETURN coefficients B and evaluation metrics

END Multiple_Linear_Regression

Output:

Estimated Coefficients:

b0 = 5.2, b1 = 0.35 (Attendance), b2 = 0.42 (Internal Marks),

b3 = 0.30 (Assignment Marks), b4 = 1.10 (Study Hours)

Regression Equation:

Final_Mark = 5.2 + 0.35*Attendance + 0.42*Internal

 + 0.30*Assignment + 1.10*Study_Hours

Model Evaluation:

$MAE = 3.12$

$RMSE = 4.05$

$R\text{-Squared} = 0.89$

The model predicts final marks with high accuracy, explaining 89% of the variation in student performance.